

Họ tên sinh viên: Hoàng Thị Minh Thu - 20252793M

BÁO CÁO: Thực hành ảo hóa IoT - Lab work:

Week 3 - Day 1: Xây dựng IoT Stack đa dịch vụ với Docker Compose, MQTT, InfluxDB và Grafana

Mục lục

I. Mục tiêu bài thực hành:	3
II. Kiến thức cần nắm:	3
III. Kiến trúc hệ thống thực hành:	3
IV. Chuẩn bị môi trường	4
1. Kiểm tra docker compose:	4
2. Tạo thư mục project:	4
V. Cấu hình Mosquitto MQTT Broker:	4
VI. Service 1: Sensor simulator:	5
1. File <code>iot-sensor-simulator/requirements.txt</code> :	5
2. File <code>iot-sensor-simulator/app.py</code> :	6
3. File <code>iot-sensor-simulator/Dockerfile</code> :	7
VII. Service 2: System monitor:	7
1. File <code>system_monitor/requirements.txt</code> :	7
2. File <code>system_monitor/app.py</code> :	8
3. File <code>system_monitor/Dockerfile</code> :	9
VIII. Service 3: Data processor:	9
1. File <code>processor/requirements.txt</code> :	9
2. File <code>processor/app.py</code> :	10
3. File <code>processor/Dockerfile</code> :	12
IX. File <code>docker_compose.yml</code>:	12
X. Khởi động IoT Stack:	13
XI. Kiểm thử MQTT:	14
XII. Kiểm tra InfluxDB:	15
XIII. Cấu hình Grafana:	16
XIV. Thêm InfluxDB Data Source:	16
XV. Tạo dashboard tối thiểu:	17
XVI. Debug và quan sát hệ thống:	19
XVII. Câu hỏi báo cáo	19
1. Docker Compose khác gì so với việc chạy nhiều lệnh <code>`docker run`</code> riêng lẻ?	19
2. Trong <code>docker-compose.yml</code> , vì sao service <code>sensor</code> dùng <code>MQTT_BROKER: mosquitto</code> thay vì <code>localhost</code> ?	20
3. Vai trò của <code>processor</code> trong IoT stack là gì?	20
4. Vì sao dữ liệu <code>sensor</code> nên được lưu vào <code>time-series database</code> ?	20
5. InfluxDB bucket là gì?	20
6. Trong Grafana, vì sao URL của InfluxDB phải là <code>http://influxdb:8086</code> ?	20
7. Docker volume <code>influxdb-data</code> có vai trò gì?	20
8. Nếu xóa container InfluxDB nhưng giữ volume, dữ liệu có mất không? Giải thích.	20
9. Nếu muốn thêm một sensor mới, em sẽ sửa hệ thống như thế nào?	20
10. Trong hệ thống thật, vì sao không nên dùng token và password đơn giản như trong bài lab?	20

XVIII. Dọn dẹp môi trường

20

XIX. Kết luận:

21

I. Mục tiêu bài thực hành:

- + Hiểu vai trò của Docker compose trong triển khai hệ thống IoT nhiều dịch vụ.
- + Xây dựng 1 IoT Stack bao gồm MQTT broker, sensor simulator, system monitor, data processor, InfluxDB và Grafana.
- + Hiểu cách các container giao tiếp với nhau thông qua Docker network nội bộ.
- + Lưu dữ liệu IoT dạng time-series vào InfluxDB .
- + Cấu hình Grafana kết nối đến InfluxDB và tạo dashboard giám sát dữ liệu.
- + Kiểm tra log, trạng thái service và debug lỗi trong Docker Compose.

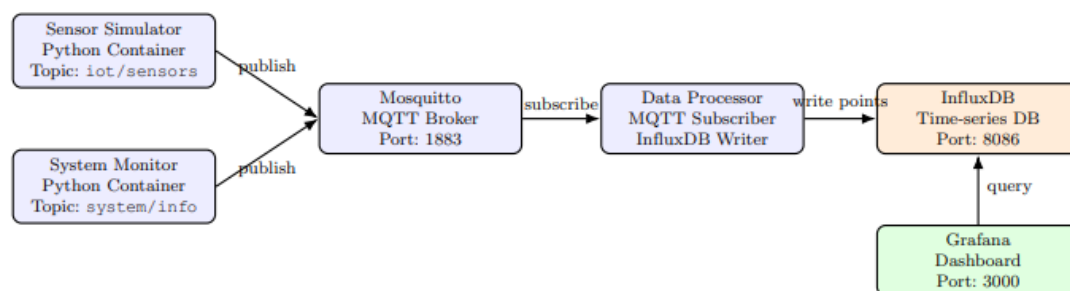
II. Kiến thức cần nắm:

Sinh viên cần nắm các kiến thức cơ bản sau:

- + Docker compose: công cụ mô tả và chạy multi-container application bằng file docker-compose.yml.
- + Service: một thành phần trong docker compose, ví dụ mosquitto, sensor, influxDB, grafana, processor.
- + Docker network: mạng ảo cho phép các container giao tiếp với nhau bằng tên service.
- + Volume: cơ chế dữ liệu bền vững, không bị mất khi container bị xóa.
- + Time series database: cơ sở dữ liệu tối ưu cho dữ liệu gắn với thời gian, ví dụ dữ liệu sensor.
- + Dashboard: giao diện trực quan hóa dữ liệu bằng biểu đồ, panel, gauge hoặc table.

III. Kiến trúc hệ thống thực hành:

- Kiến trúc:



Hình 1: Kiến trúc IoT stack đa dịch vụ trong bài lab Buổi 2

- Giải thích thành phần:

- + sensor: gửi dữ liệu nhiệt độ và độ ẩm lên topic iot/sensors.
- + system-monitor: gửi thông tin cpu, ram, disk lên topic system/ info.
- + processor: subscribe dữ liệu từ MQTT broker, parse JSON và ghi dữ liệu vào influxDB.
- + Grafana: kết nối dữ liệu đến influxDB để hiển thị dashboard.

- Từ docker run đến docker compose:

- + Trong buổi 1 chúng ta chạy từng container với docker run, cách này giúp ta hiểu rõ từng tham số nhưng sẽ bất tiện khi hệ thống có nhiều container.
- + Docker giải quyết vấn đề này bằng cách mô tả hệ thống bằng 1 file duy nhất: dockercompose.yml. Sau đó ta có thể khởi động hệ thống bằng lệnh: docker compose up -d --build.


```
🔧 mosquitto.conf X
mosquitto > config > 🔧 mosquitto.conf
1 listener 1883
2 allow_anonymous true
```

VI. Service 1: Sensor simulator:

- Service này sinh dữ liệu, tạo độ ẩm giả lập rồi publish lên hệ thống iot/sensors.

1. File **iot-sensor-simulator/requirements.txt**:

- Để [app.py](#) có thể nói chuyện với mosquitto mqtt thì cần tới thư viện paho-mqtt

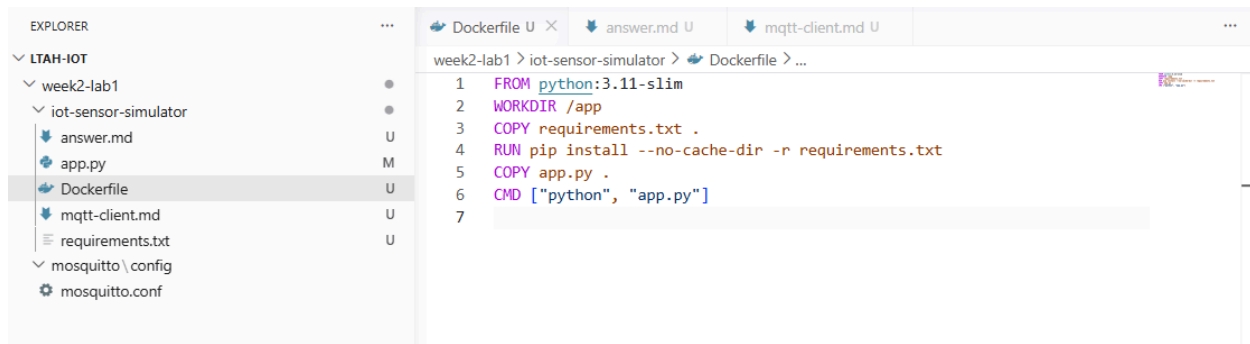
```
EXPLORER
▼ LTAH-IOT
  ▼ week2-lab1
    ▼ iot-sensor-simulator
      app.py
      dockercompose.yml
      requirements.txt
  ▼ mosquitto \ config
    🔧 mosquitto.conf

requirements.txt X
week2-lab1 > iot-sensor-simulator > requirements.txt
1 paho-mqtt==1.6.1
```

2. File `iot-sensor-simulator/app.py`:

```
1 import json
2 import os
3 import random
4 import time
5 from datetime import datetime, timezone
6 #datetime, timezone nằm trong module datetime của python
7
8 import paho.mqtt.client as mqtt
9
10 #os.getenv là hàm trong module os, dùng để đọc giá trị của 1 biến môi trường trong hệ điều hành
11 #cú pháp: os.getenv('TÊN BIẾN', 'giá trị mặc định');
12 #biến môi trường là gì? là các giá trị cấu hình được lưu bên ngoài code, ở cấp hệ điều hành
13 #vd: trước khi chạy chương trình, bạn có thể đặt
14 #trên terminal (Linux/Mac): export MQTT_BROKER = 192.168.1.100; Windows: $env:MQTT_BROKER = '192.169.1.100'
15 #khi đó os.getenv('MQTT_BROKER', 'localhost') sẽ trả về 192.168.1.100 thay vì localhost
16 MQTT_BROKER = os.getenv('MQTT_BROKER', 'localhost') #địa chỉ của mqtt_broker: mặc định là localhost
17 MQTT_PORT = int(os.getenv('MQTT_PORT', '1883')) #os.getenv trả về string, sử dụng int(os.getenv...) giúp giá trị trả về dưới dạng int
18 MQTT_TOPIC = os.getenv('MQTT_TOPIC', 'iot/sensors')
19 DEVICE_ID = os.getenv('DEVICE_ID', 'sensor-01')
20 PUBLISH_INTERVAL = float(os.getenv('PUBLISH_INTERVAL', '2'))
21
22 #Trong python, sau khi khai báo hàm với dấu : ở cuối,
23 #tất cả các dòng thuộc thân hàm phải được thụt vào trong (thường là 4 dấu cách)
24 def create_sensor_data():
25     #tạo giá trị mô phỏng cho sensor
26     temperature = round(random.uniform(25.0,35.0),2)
27     humidity = round(random.uniform(50.0,90.0),2)
28     #uniform chỉ nhận 2 tham số còn 2 là của round
29
30     #trong dictionary (từ điển) phải dùng dấu , chứ không phải dấu ;
31     #dictionary là 1 kiểu dữ liệu trong Python để lưu trữ dữ liệu theo dạng cặp key: value
32     data = {
33         'device_id': DEVICE_ID,
34         'temperature': temperature,
35         'humidity': humidity,
36         'timestamp': datetime.now(timezone.utc).isoformat()
37     }
38
39     return data
40
41 def main(): #định nghĩa hàm main, là hàm điều chỉnh toàn bộ luồng chạy của chương trình (kết nối -> tạo dữ liệu -> gửi đi)
42     #tạo 1 mqtt client
43     client = mqtt.Client(client_id=DEVICE_ID)
44     #in thông báo ra màn hình nhờ f-string - cho phép chèn giá trị biến vào trong {...}
45     print(f'Connecting to mqtt broker at {MQTT_BROKER}:{MQTT_PORT}...')
46     #thực hiện kết nối tới broker, mở kết nối 60s, nếu trong 60s không có dữ liệu trao đổi, client sẽ ping để báo 'tôi vẫn sống',
47     #nếu broker không nhận được -> coi như client mất kết nối
48     client.connect(MQTT_BROKER, MQTT_PORT, keepalive=60)
49
50     print(f"Publishing data to topic: {MQTT_TOPIC}")
51
52     while True:
53         data = create_sensor_data(1)
54         payload = json.dumps(data)
55
56         result = client.publish(MQTT_TOPIC, payload)
57         if result.rc == mqtt.MQTT_ERR_SUCCESS:
58             print(f'Publisher: {payload}')
59         else:
60             print(f'Failed to publish message. Error code: {result.rc}')
61
62         time.sleep(PUBLISH_INTERVAL)
63
64 if __name__ == '__main__':
65     main()
66
```

3. File `iot-sensor-simulator/Dockerfile`:



```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY app.py .
6 CMD ["python", "app.py"]
7
```

VII. Service 2: System monitor:

- Service này thu thập thông tin hệ thống như CPU, RAM, disk vào publish lên topic `system/info`.

1. File `system_monitor/requirements.txt`:

- Để [app.py](#) có thể nói chuyện với mosquitto mqtt thì cần tới thư viện `paho-mqtt`.
- Psutil để lấy thông tin về hệ thống và các tiến trình đang chạy.



```
1 paho-mqtt==1.6.1
2 psutil==5.9.8
```


2. File system_monitor/app.py:

week2-lab1 > system_monitor > system_app.py > connect_mqtt_client

```
1 import json
2 import os
3 import socket
4 import time
5 from datetime import datetime, timezone
6
7 import paho.mqtt as mqtt
8 import psutil
9
10 #khai báo giá trị mặc định cho các biến môi trường
11 MQTT_BROKER = os.getenv('MQTT_BROKER', 'localhost')
12 MQTT_PORT = int(os.getenv('MQTT_PORT', '1883'))
13 MQTT_TOPIC = os.getenv('MQTT_TOPIC', 'system/info')
14 DEVICE_ID = os.getenv('DEVICE_ID', 'system-monitor-01')
15 PUBLISH_INTERVAL = float(os.getenv('PUBLISH_INTERVAL', '5'))
16
17 #thu thập thông tin hệ thống hiện tại bằng thư viện psutil, đóng gói thành 1 dict json
18 def create_system_info():
19     cpu_percent = psutil.cpu_percent(interval=1)
20     memory = psutil.virtual_memory()
21     disk=psutil.disk_usage("/")
22
23     return {
24         "device_id": DEVICE_ID,
25         "hostname": socket.gethostname(),
26         "cpu_percent": cpu_percent,
27         "memory_percent": memory.percent,
28         "memory_used_mb": round(memory.used / (1024 * 1024), 2),
29         "memory_total_mb": round(memory.total / (1024 * 1024), 2),
30         "disk_percent": disk.percent,
31         "disk_used_gb": round(disk.used / (1024 * 1024 * 1024), 2),
32         "disk_total_gb": round(disk.total / (1024 * 1024 * 1024), 2),
33         "timestamp": datetime.now(timezone.utc).isoformat()
34     }
35
36 #tạo mqtt_client và kết nối đến broker. Nếu kết nối thất bại thì tự động retry trong 5s cho đến khi thành công
37 #trả về client đã kết nối
38 def connect_mqtt_client():
39     client = mqtt.Client(client_id = DEVICE_ID)
40
41 #giải thích while True: try except:
42 #code trong try chạy bình thường -> except bị bỏ qua, code trong try gặp lỗi -> nhảy xuống except để xử lý
43 while True:
44     try:
45         print(f'connecting to MQTT Broker at {MQTT_BROKER}:{MQTT_PORT}...')
46         client.connect(MQTT_BROKER, MQTT_PORT, keepalive = 60)
47         print('Connected to Mqtt Broker.')
48         return client
49
50     except Exception as exc:
51         print(f'Connection failed: {exc}')
52         print('retrying in 5 seconds...')
53         time.sleep(5)
54
55 #vòng lặp chính của chương trình, kết nối mqtt broker. cứ mỗi publish_interval giây (mặc định 5s)
56 #gọi create_system_info()->serialize json (chuyển đổi dữ liệu (object/dict) thành chuỗi văn bản JSON) -> publish lên topic mqtt
57 #nếu xảy ra lỗi runtime, tự động reconnect
58 def main():
59     client = connect_mqtt_client()
60     print(f'Connecting to Mqtt broker at {MQTT_BROKER}:{MQTT_PORT}...')
61
62     while True:
63         try:
64             data = create_system_info()
65             payload = json.dumps(data)
66             result = client.publish(MQTT_TOPIC, payload)
67
68             if result.rc == mqtt.MQTT_ERR_SUCCESS:
69                 print(f'Published: {payload}')
70             else: print(f'Failed to publish message. Error code: {result.rc}')
71             time.sleep(PUBLISH_INTERVAL)
72         except Exception as exc:
73             print(f'Runtime error: {exc}')
74             client = connect_mqtt_client()
75
76 if __name__ == '__main__':
77     main()
```

3. File system_monitor/Dockerfile:

```
week2-lab1 > system_monitor > Dockerfile > ...
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir \
7     paho-mqtt==1.6.1 \
8     psutil==5.9.8
9
10 COPY system_app.py .
11
12 #khai báo các biến môi trường
13 ENV MQTT_BROKER=localhost \
14     MQTT_PORT=1883 \
15     MQTT_TOPIC=system/info \
16     DEVICE_ID=system-monitor-01 \
17     PUBLISH_INTERVAL=5
18
19 CMD ["python", "system_app.py"]
20
```

VIII. Service 3: Data processor:

- Service processor nhận data từ MQTT Broker và ghi vào InfluxDB.

1. File processor/requirements.txt:

```
week2-lab1 > processor > requirements.txt
1 paho-mqtt==1.6.1
2 influxdb-client==1.43.0
```

2. File processor/app.py:

```
1 import json
2 import os
3 import time
4 from datetime import timezone, datetime
5
6 import paho.mqtt.client as mqtt
7 from influxdb_client import InfluxDBClient, Point, WritePrecision
8 #processor - mqtt -> influxdb: nhận message từ mqtt rồi lưu vào db influxdb
9
10 #khai báo giá trị mặc định của biến môi trường
11 MQTT_BROKER = os.getenv('MQTT_BROKER', 'localhost')
12 MQTT_PORT = int(os.getenv('MQTT_PORT', '1883'))
13 #topic # - lắng nghe toàn bộ message từ docker
14 MQTT_TOPIC = os.getenv('MQTT_TOPIC', '#')
15
16 #khai báo biến liên quan influx db
17 INFLUXDB_URL = os.getenv('INFLUXDB_URL', 'http://influxdb:8086')
18 INFLUXDB_TOKEN = os.getenv('INFLUXDB_TOKEN', 'iot-token-123')
19 INFLUXDB_ORG = os.getenv('INFLUXDB_ORG', 'iot-lab')
20 INFLUXDB_BUCKET = os.getenv('INFLUXDB_BUCKET', 'iot_data')
21
22 #chuyển đổi chuỗi timestamp (2024-01-01T00:00:00Z) thành object datetime
23 #trả về none nếu chuỗi rỗng hoặc lỗi
24 def parse_timestamp(timestamp_text):
25     if not timestamp_text:
26         return None
27     try:
28         normalized = timestamp_text.replace('Z', '+00:00')
29         return datetime.fromisoformat(normalized)
30     except Exception:
31         return None
32
33 #ánh xạ tên mqtt topic sang tên bảng trong influxDB
34 def get_measurement_name(topic):
35     if topic == 'iot/sensors':
36         return 'sensor_data'
37     if topic == 'system/info':
38         return 'system_info'
39     return 'mqtt_data'
40
41 #chuyển đổi 1 message json sang 1 influxDB Point (Định dạng để ghi vào db)
42 #gắn tag device_id và topic, duyệt qua từng field trong data để thêm vào point, trả về point nếu k có field hợp lệ
43 def create_point(topic, data):
44     measurement = get_measurement_name(topic)
45     device_id = str(data.get('device_id', 'unknown'))
```



```

47 point = Point(measurement)
48 point = point.tag('device_id', device_id)
49 point = point.tag('topic', topic)
50
51 timestamp = parse_timestamp(data.get('timestamp'))
52 if timestamp is not None:
53     point = point.time(timestamp, WritePrecision.NS)
54
55 field_count = 0
56
57 for key, value in data.items():
58     if key in ['device_id', 'timestamp']:
59         continue
60     if isinstance(value, bool):
61         point = point.field(key, value)
62         field_count += 1
63     elif isinstance(value, int):
64         point = point.field(key, value)
65         field_count += 1
66     elif isinstance(value, float):
67         point = point.field(key, value)
68         field_count += 1
69     elif isinstance(value, str):
70         point = point.field(key, value)
71         field_count += 1
72
73 if field_count == 0:
74     return None
75 return point
76
77 #cổng kết nối đến influxdb - retry mỗi 5s
78 def connect_influxdb():
79     while True:
80         try:
81             client = InfluxDBClient(
82                 url = INFLUXDB_URL,
83                 token = INFLUXDB_TOKEN,
84                 org=INFLUXDB_ORG
85             )
86             health = client.health()
87             print(f'InfluxDB health: {health.status}')
88             write_api = client.write_api()
89             return write_api
90
91         except Exception as exc:
92             print(f'Cannot connect to InfluxDB: {exc}')
93             print('Retrying in 5 seconds ...')
94             time.sleep(5)
95
96 #write_api để ghi dữ liệu
97 write_api = connect_influxdb()
98
99 #callback được gọi mỗi khi mqtt kết nối thành công, subscribe vào topic đã cấu hình (#)
100 def on_connect(client, userdata, flags, rc):
101     if rc == 0:
102         print('Connected to MQTT broker.')
103         client.subscribe(MQTT_TOPIC)
104         print(f'Subscribed to topic: {MQTT_TOPIC}')
105     else:
106         print(f'Failed to connect to MQTT broker. Return code: {rc}')
107
108 #callback được gọi mỗi khi nhận message mqtt
109 #giải mã json -> tạo influxDB point -> ghi vào db
110 def on_message(client, userdata, message):
111     topic = message.topic
112
113     try:
114         payload_text = message.payload.decode('utf-8')
115         data = json.loads(payload_text)
116         point = create_point(topic, data)
117
118         if point is None:
119             print(f'Skipped message without valid fields: {payload_text}')
120             return
121
122         write_api.write(
123             bucket=INFLUXDB_BUCKET,
124             org=INFLUXDB_ORG,
125             record=point
126         )
127
128         print(f'Written to InfluxDB: topic = {topic}, payload={payload_text}')
129
130     except Exception as exc:
131         print(f'Failed to process message from topic {topic}:{exc}')

```



```

131 def main():
132     mqtt_client = mqtt.Client(client_id = 'iot-data-processor')
133     mqtt_client.on_connect = on_connect #khai báo callback mỗi khi connect thành công
134     mqtt_client.on_message = on_message #khai báo callback mỗi khi gửi message thành công
135
136     #khởi tạo mqtt client, kết nối broker -> chạy loop_forever() để nhận message liên tục
137     while True:
138         try:
139             print(f'Connecting to MQTT broker at {MQTT_BROKER}:{MQTT_PORT} ...')
140             mqtt_client.connect(MQTT_BROKER, MQTT_PORT, keepalive = 60)
141             mqtt_client.loop_forever()
142         except Exception as exc:
143             print(f'MQTT connection error: {exc}')
144             print('Retrying in 5 seconds')
145             time.sleep(5)
146
147 if __name__ == '__main__':
148     main()

```

3. File processor/Dockerfile:

week2-lab1 > processor > Dockerfile > ...

```

1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6 RUN pip install --no-cache-dir -r requirements.txt
7 COPY processor.py .
8
9 CMD ["python", "processor.py"]
10

```

IX. File docker_compose.yml:

- Docker-compose.yml là file khai báo cấu hình — nói cho Docker biết cần chạy container nào, dùng image gì, biến môi trường và thứ tự khởi động ra sao. Bản thân file không chạy gì cả, Docker đọc rồi tự lo phần còn lại

```

1 services:
2     # MQTT Broker - trung gian nhận và phân phối message
3     > Run Service
4     mosquitto:
5         image: eclipse-mosquitto:2
6         ports:
7             - "1883:1883"
8         volumes:
9             - ./mosquitto/config/mosquitto.conf:/mosquitto/config/mosquitto.conf
10
11     # Database lưu trữ time-series data
12     > Run Service
13     influxdb:
14         image: influxdb:2.7
15         ports:
16             - "8086:8086"
17         environment:
18             - DOCKER_INFLUXDB_INIT_MODE=setup
19             - DOCKER_INFLUXDB_INIT_USERNAME=admin
20             - DOCKER_INFLUXDB_INIT_PASSWORD=admin123
21             - DOCKER_INFLUXDB_INIT_ORG=iot-lab
22             - DOCKER_INFLUXDB_INIT_BUCKET=iot_data
23             - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=iot-token-123
24         volumes:
25             - influxdb_data:/var/lib/influxdb2
26
27     # Giả lập cảm biến IoT, publish data lên MQTT
28     > Run Service
29     iot-sensor-simulator:
30         build: ./iot-sensor-simulator
31         environment:
32             - MQTT_BROKER=mosquitto
33             - MQTT_PORT=1883
34         depends_on:
35             - mosquitto

```

```

33
34 # Giám sát tài nguyên hệ thống (CPU, RAM, disk), publish lên MQTT
D-Run Service
35 system-monitor:
36   build: ./system_monitor
37   environment:
38     - MQTT_BROKER=mosquitto
39     - MQTT_PORT=1883
40     - MQTT_TOPIC=system/info
41     - DEVICE_ID=system-monitor-01
42     - PUBLISH_INTERVAL=5
43   #depends_on đảm bảo thứ tự khởi động
44   depends_on:
45     - mosquitto
46
47 # Nhận message từ MQTT, xử lý và lưu vào InfluxDB
D-Run Service
48 processor:
49   build: ./processor
50   environment:
51     - MQTT_BROKER=mosquitto
52     - MQTT_PORT=1883
53     - MQTT_TOPIC=#
54     - INFLUXDB_URL=http://influxdb:8086
55     - INFLUXDB_TOKEN=iot-token-123
56     - INFLUXDB_ORG=iot-lab
57     - INFLUXDB_BUCKET=iot_data
58   depends_on:
59     - mosquitto
60     - influxdb
61

```

```

grafana:
  image: grafana/grafana:10.4.3
  ports:
    - "3000:3000"
  environment:
    - GF_SECURITY_ADMIN_USER=admin
    - GF_SECURITY_ADMIN_PASSWORD=admin
  volumes:
    - grafana_data:/var/lib/grafana
  depends_on:
    - influxdb

```

```

volumes:
  influxdb_data:
  grafana_data:

```

X. Khởi động IoT Stack:

- Chạy docker compose: `docker compose up -d --build` -> chạy thành công 6 services

```

PS C:\Users\ACER\Desktop\LTAH-IoT\week2-lab1> docker compose up -d --build
#28 DONE 0.1s

[+] up 21/21or] resolving provenance for metadata file
✓ Image grafana/grafana:10.4.3          Pulled          12.4s
✓ Image week2-lab1-processor            Built           2.8s
✓ Image week2-lab1-iot-sensor-simulator Built           2.8s
✓ Image week2-lab1-system-monitor       Built           2.8s
✓ Volume week2-lab1_grafana_data         Created         0.0s
✓ Container week2-lab1-mosquitto-1       Running         0.0s
✓ Container week2-lab1-influxdb-1        Running         0.0s
✓ Container week2-lab1-grafana-1         Started         2.8s
✓ Container week2-lab1-system-monitor-1  Started         2.6s
✓ Container week2-lab1-iot-sensor-simulator-1 Started         2.6s
✓ Container week2-lab1-processor-1       Started         2.6s

What's next:
  Filter, search, and stream logs from all your Compose services
  in one place with Docker Desktop's Logs view. docker-desktop:///dashboard/logs
PS C:\Users\ACER\Desktop\LTAH-IoT\week2-lab1>

```

- Kiểm tra trạng thái services: `docker compose logs -f`

```

Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 31.41, "humidity": 65.05, "timestamp": "2026-06-21T14:21:51.860613+00:00"}
Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 26.14, "humidity": 57.56, "timestamp": "2026-06-21T14:21:53.860913+00:00"}
Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 27.76, "humidity": 84.39, "timestamp": "2026-06-21T14:21:55.861453+00:00"}
Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 34.49, "humidity": 52.2, "timestamp": "2026-06-21T14:21:57.862166+00:00"}
1782051720: New connection from 172.19.0.4:59915 on port 1883.
1782051720: Client system-monitor-01 [172.19.0.4:34439] disconnected: connection closed by client.
1782051720: New client connected from 172.19.0.4:59915 as system-monitor-01 (p4, c1, k60).
1782051721: New connection from 172.19.0.4:51889 on port 1883.
1782051721: Client system-monitor-01 [172.19.0.4:59915] disconnected: connection closed by client.
1782051721: New client connected from 172.19.0.4:51889 as system-monitor-01 (p4, c1, k60).
1782051722: New connection from 172.19.0.4:57877 on port 1883.
1782051722: Client system-monitor-01 [172.19.0.4:51889] disconnected: connection closed by client.
1782051722: New client connected from 172.19.0.4:57877 as system-monitor-01 (p4, c1, k60).
1782051723: New connection from 172.19.0.4:50545 on port 1883.
1782051723: Client system-monitor-01 [172.19.0.4:57877] disconnected: connection closed by client.
1782051723: New client connected from 172.19.0.4:50545 as system-monitor-01 (p4, c1, k60).
1782051724: New connection from 172.19.0.4:47779 on port 1883.
1782051724: Client system-monitor-01 [172.19.0.4:50545] disconnected: connection closed by client.
1782051724: New client connected from 172.19.0.4:47779 as system-monitor-01 (p4, c1, k60).
1782051725: New connection from 172.19.0.4:59021 on port 1883.

```

- Kiểm tra log từng service: `docker compose logs -f <<tên service>>`

+ mosquitto:

```
mosquitto-1 | 1782053207: New client connected from 172.19.0.4:33981 as system-monitor-01 (p4, c1, k60).
mosquitto-1 | 1782053208: New connection from 172.19.0.4:49939 on port 1883.
mosquitto-1 | 1782053208: Client system-monitor-01 [172.19.0.4:33981] disconnected: connection closed by client.
mosquitto-1 | 1782053208: New client connected from 172.19.0.4:49939 as system-monitor-01 (p4, c1, k60).
mosquitto-1 | 1782053209: New connection from 172.19.0.4:34061 on port 1883.
mosquitto-1 | 1782053209: Client system-monitor-01 [172.19.0.4:49939] disconnected: connection closed by client.
mosquitto-1 | 1782053209: New client connected from 172.19.0.4:34061 as system-monitor-01 (p4, c1, k60).
mosquitto-1 | 1782053210: New connection from 172.19.0.4:33155 on port 1883.
mosquitto-1 | 1782053210: Client system-monitor-01 [172.19.0.4:34061] disconnected: connection closed by client.
mosquitto-1 | 1782053210: New client connected from 172.19.0.4:33155 as system-monitor-01 (p4, c1, k60).
mosquitto-1 | 1782053211: New connection from 172.19.0.4:46845 on port 1883.
mosquitto-1 | 1782053211: Client system-monitor-01 [172.19.0.4:33155] disconnected: connection closed by client.
mosquitto-1 | 1782053211: New client connected from 172.19.0.4:46845 as system-monitor-01 (p4, c1, k60).
mosquitto-1 | 1782053212: New connection from 172.19.0.4:57965 on port 1883.
mosquitto-1 | 1782053212: Client system-monitor-01 [172.19.0.4:46845] disconnected: connection closed by client.
mosquitto-1 | 1782053212: New client connected from 172.19.0.4:57965 as system-monitor-01 (p4, c1, k60).
mosquitto-1 | 1782053213: New connection from 172.19.0.4:52863 on port 1883.
mosquitto-1 | 1782053213: Client system-monitor-01 [172.19.0.4:57965] disconnected: connection closed by client.
```

+ system-monitor:

```
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
system-monitor-1 | connecting to MQTT Broker at mosquitto:1883...
system-monitor-1 | Connected to Mqtt Broker.
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
system-monitor-1 | connecting to MQTT Broker at mosquitto:1883...
system-monitor-1 | Connected to Mqtt Broker.
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
system-monitor-1 | connecting to MQTT Broker at mosquitto:1883...
system-monitor-1 | Connected to Mqtt Broker.
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
system-monitor-1 | connecting to MQTT Broker at mosquitto:1883...
system-monitor-1 | Connected to Mqtt Broker.
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
system-monitor-1 | connecting to MQTT Broker at mosquitto:1883...
system-monitor-1 | Connected to Mqtt Broker.
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
system-monitor-1 | connecting to MQTT Broker at mosquitto:1883...
system-monitor-1 | Connected to Mqtt Broker.
system-monitor-1 | Runtime error: module 'json' has no attribute 'dumbs'
```

+ iot-sensor-simulator:

```
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 32.38, "humidity": 68.62, "timestamp": "2026-06-21T14:53:26.344454+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 26.96, "humidity": 64.16, "timestamp": "2026-06-21T14:53:28.344960+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 34.57, "humidity": 85.0, "timestamp": "2026-06-21T14:53:30.345675+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 27.61, "humidity": 60.08, "timestamp": "2026-06-21T14:53:32.346424+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 30.87, "humidity": 84.81, "timestamp": "2026-06-21T14:53:34.346926+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 25.09, "humidity": 66.73, "timestamp": "2026-06-21T14:53:36.347567+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 34.12, "humidity": 87.52, "timestamp": "2026-06-21T14:53:38.348061+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 34.76, "humidity": 88.87, "timestamp": "2026-06-21T14:53:40.348901+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 33.37, "humidity": 57.51, "timestamp": "2026-06-21T14:53:42.349591+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 26.53, "humidity": 74.47, "timestamp": "2026-06-21T14:53:44.349887+00:00"}
iot-sensor-simulator-1 | Publisher: {"device_id": "sensor-01", "temperature": 30.7, "humidity": 80.6, "timestamp": "2026-06-21T14:53:46.350187+00:00"}
```

+ processor:

```
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 34.83, "humidity": 76.49, "timestamp": "2026-06-21T14:53:10.342225+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 31.26, "humidity": 89.42, "timestamp": "2026-06-21T14:53:12.342975+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 27.44, "humidity": 52.91, "timestamp": "2026-06-21T14:53:14.343496+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 31.13, "humidity": 83.45, "timestamp": "2026-06-21T14:53:16.344059+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 33.48, "humidity": 51.06, "timestamp": "2026-06-21T14:53:18.344288+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 32.63, "humidity": 76.24, "timestamp": "2026-06-21T14:53:20.347234+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 33.06, "humidity": 67.9, "timestamp": "2026-06-21T14:53:22.343313+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 32.22, "humidity": 80.18, "timestamp": "2026-06-21T14:53:24.343855+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 32.38, "humidity": 68.62, "timestamp": "2026-06-21T14:53:26.344454+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 26.96, "humidity": 64.16, "timestamp": "2026-06-21T14:53:28.344960+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 34.57, "humidity": 85.0, "timestamp": "2026-06-21T14:53:30.345675+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 27.61, "humidity": 60.08, "timestamp": "2026-06-21T14:53:32.346424+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 30.87, "humidity": 84.81, "timestamp": "2026-06-21T14:53:34.346926+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 25.09, "humidity": 66.73, "timestamp": "2026-06-21T14:53:36.347567+00:00"}
processor-1 | Written to InfluxDB: topic = iot/sensors, payload={"device_id": "sensor-01", "temperature": 34.12, "humidity": 87.52, "timestamp": "2026-06-21T14:53:38.348061+00:00"}
```

XI. Kiểm thử MQTT:

- Kiểm tra dữ liệu của từng topic: `mosquitto_sub -h localhost -p 1883 -t <<tên topic>>`

+ iot/sensors:

```
thu_ily@THU-ILY0508:/mnt/c/Users/ACER$ mosquitto_sub -h localhost -p 1883 -t 'iot/sensors'
{"device_id": "sensor-01", "temperature": 25.7, "humidity": 75.47, "timestamp": "2026-06-21T15:04:40.495134+00:00"}
{"device_id": "sensor-01", "temperature": 31.95, "humidity": 87.18, "timestamp": "2026-06-21T15:04:42.495769+00:00"}
{"device_id": "sensor-01", "temperature": 29.86, "humidity": 64.0, "timestamp": "2026-06-21T15:04:44.496260+00:00"}
{"device_id": "sensor-01", "temperature": 33.62, "humidity": 55.91, "timestamp": "2026-06-21T15:04:46.496815+00:00"}
```

+ system/info:

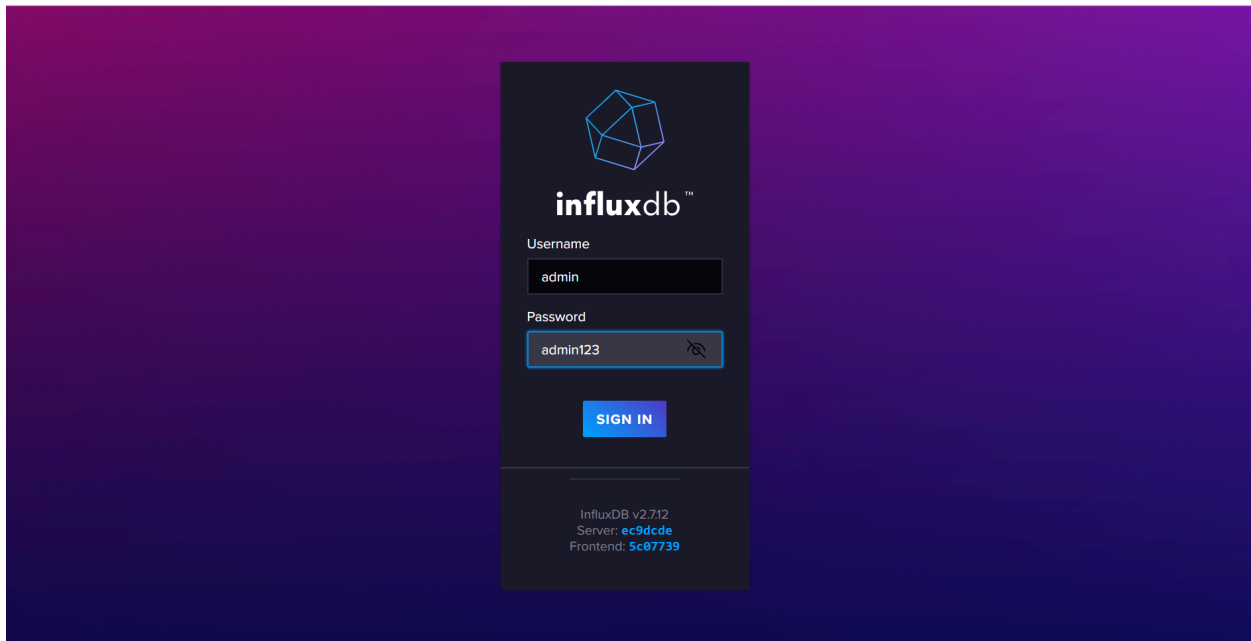
```
^Cthu_ily@THU-ILY0508:/mnt/c/Users/ACER$ mosquitto_sub -h localhost -p 1883 -t 'system/info'
{"device_id": "system-monitor-01", "hostname": "8619fcdea3ab", "cpu_percent": 11.8, "memory_percent": 16.4, "memory_used_mb": 1408.84, "memory_total_mb": 9833.62, "disk_percent": 0.9, "disk_used_gb": 8.94, "disk_total_gb": 1006.85, "timestamp": "2026-06-21T15:07:00.081290+00:00"}
{"device_id": "system-monitor-01", "hostname": "8619fcdea3ab", "cpu_percent": 0.8, "memory_percent": 16.4, "memory_used_mb": 1409.1, "memory_total_mb": 9833.62, "disk_percent": 0.9, "disk_used_gb": 8.94, "disk_total_gb": 1006.85, "timestamp": "2026-06-21T15:07:06.082124+00:00"}
{"device_id": "system-monitor-01", "hostname": "8619fcdea3ab", "cpu_percent": 0.9, "memory_percent": 16.5, "memory_used_mb": 1413.98, "memory_total_mb": 9833.62, "disk_percent": 0.9, "disk_used_gb": 8.94, "disk_total_gb": 1006.85, "timestamp": "2026-06-21T15:07:12.083406+00:00"}
```

+ Tất cả topic:

```
^Cthu_ily@THU-ILY0508:/mnt/c/Users/ACER$ mosquitto_sub -h localhost -p 1883 -t '#'
{"device_id": "sensor-01", "temperature": 31.95, "humidity": 64.67, "timestamp": "2026-06-21T15:07:34.534332+00:00"}
{"device_id": "system-monitor-01", "hostname": "8619fcdea3ab", "cpu_percent": 1.5, "memory_percent": 16.5, "memory_used_mb": 1420.39, "memory_total_mb": 9833.62, "disk_percent": 0.9, "disk_used_gb": 8.94, "disk_total_gb": 1006.85, "timestamp": "2026-06-21T15:07:36.087339+00:00"}
{"device_id": "sensor-01", "temperature": 26.68, "humidity": 78.0, "timestamp": "2026-06-21T15:07:36.534848+00:00"}
{"device_id": "sensor-01", "temperature": 33.83, "humidity": 72.2, "timestamp": "2026-06-21T15:07:38.535517+00:00"}
{"device_id": "sensor-01", "temperature": 33.28, "humidity": 61.61, "timestamp": "2026-06-21T15:07:40.536123+00:00"}
{"device_id": "system-monitor-01", "hostname": "8619fcdea3ab", "cpu_percent": 1.4, "memory_percent": 16.5, "memory_used_mb": 1421.8, "memory_total_mb": 9833.62, "disk_percent": 0.9, "disk_used_gb": 8.94, "disk_total_gb": 1006.85, "timestamp": "2026-06-21T15:07:42.089238+00:00"}
{"device_id": "sensor-01", "temperature": 26.03, "humidity": 74.11, "timestamp": "2026-06-21T15:07:42.536822+00:00"}
```

XII. Kiểm tra InfluxDB:

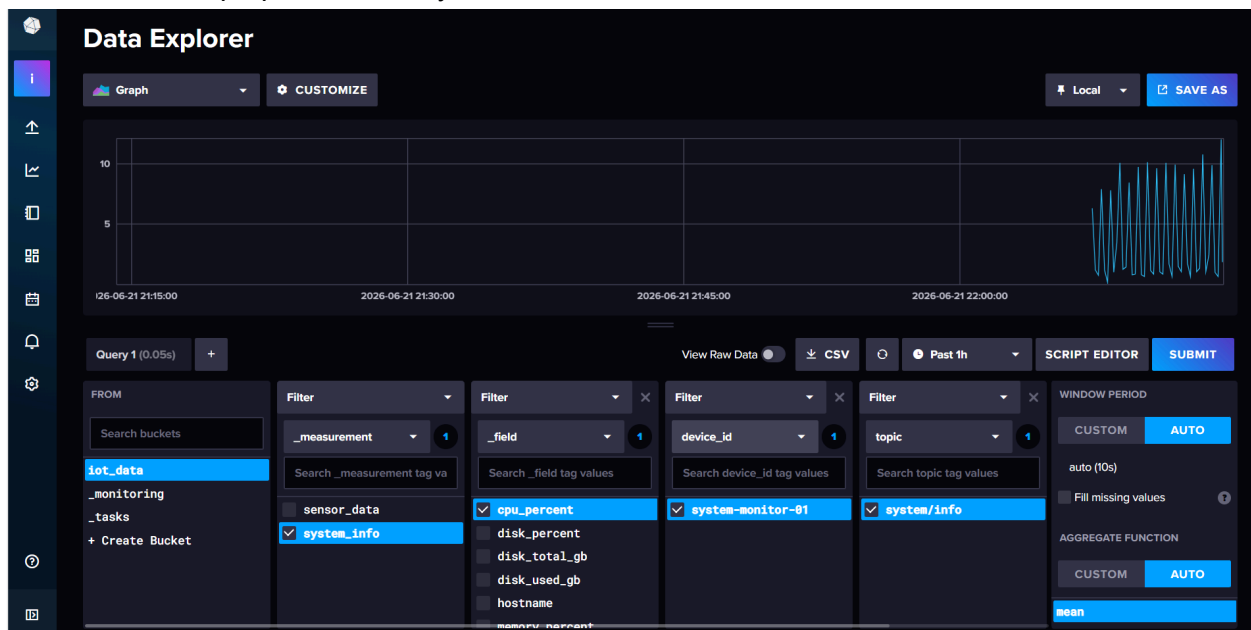
- Đăng nhập vào InfluxDB (localhost:8086 - mật khẩu tài khoản để trong file docker-compose.yml):



- Biểu đồ biến thiên nhiệt độ và độ ẩm của sensor-01:

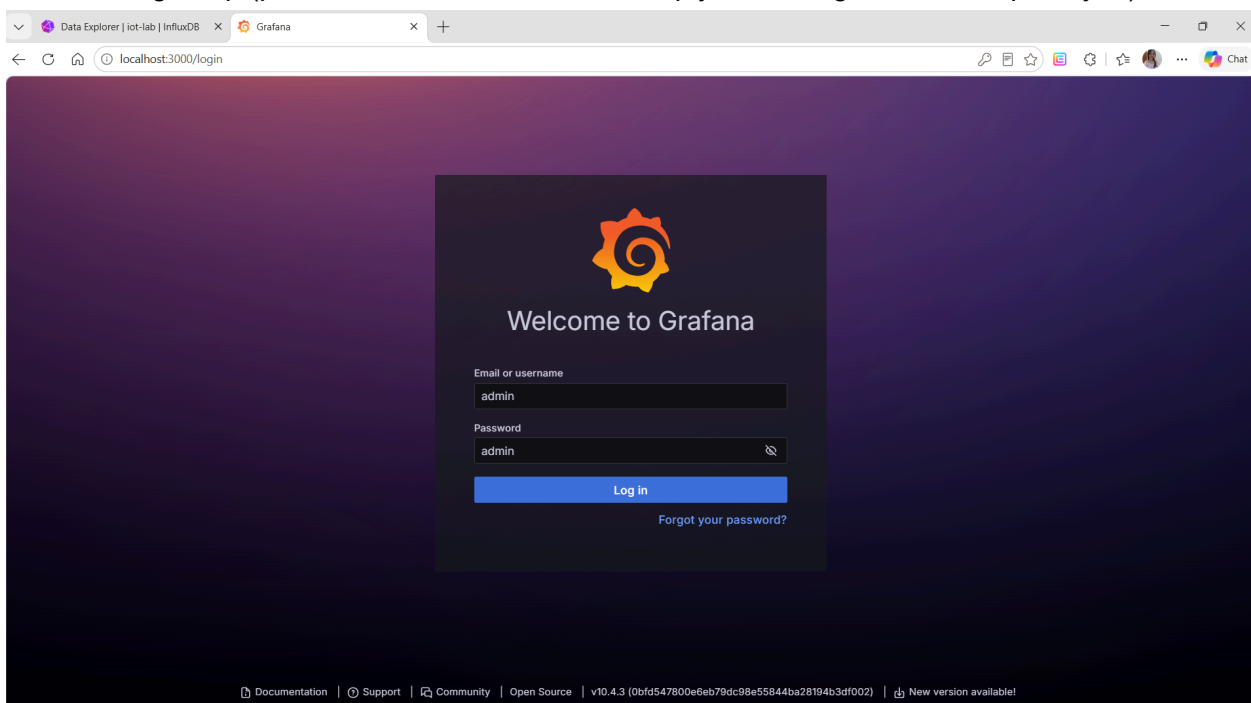


- Biểu đồ cpu percent của system monitor:



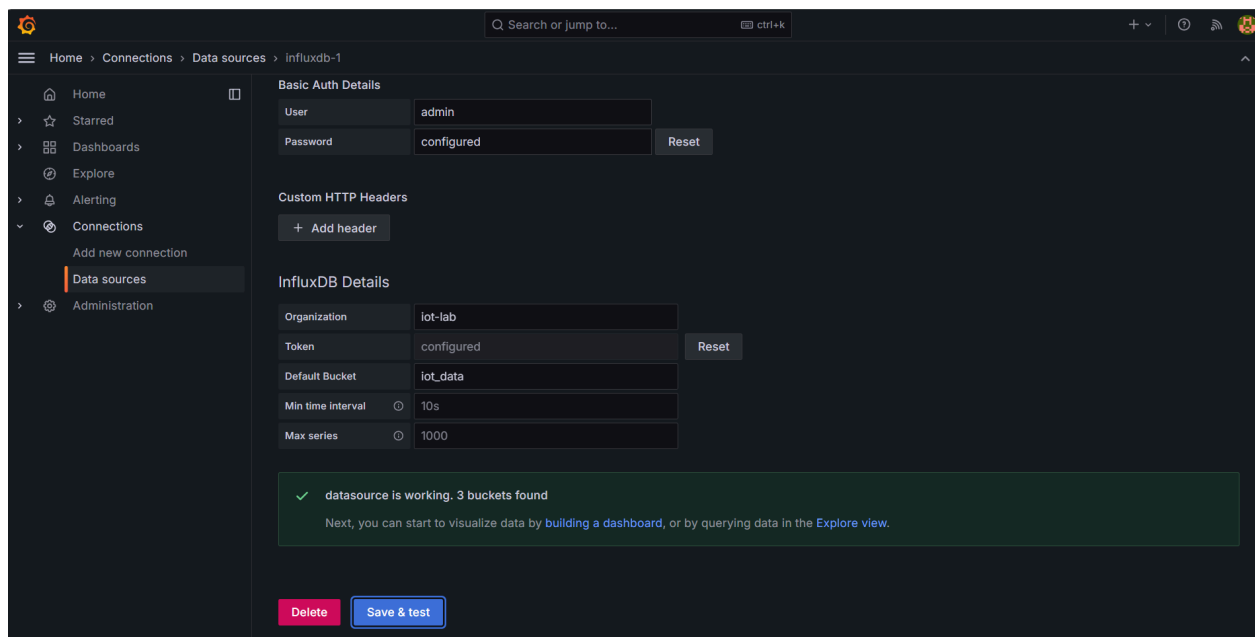
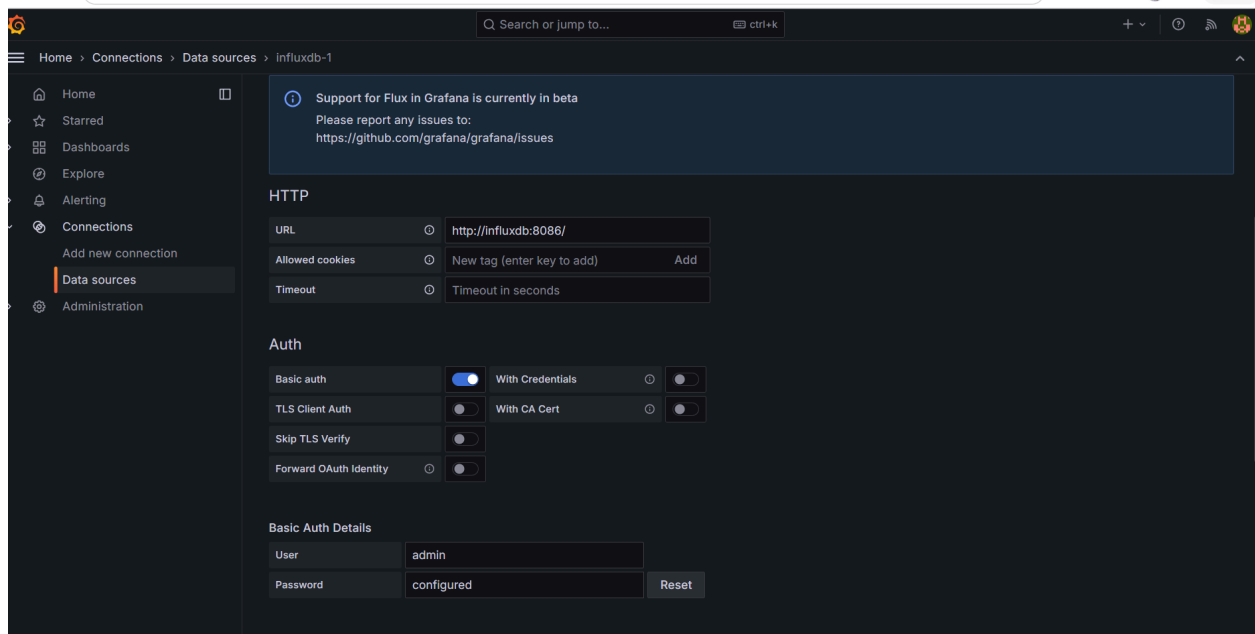
XIII. Cấu hình Grafana:

- Đăng nhập (port 3000 - mật khẩu tài khoản quy ước trong docker-compose.yml):



XIV. Thêm InfluxDB Data Source:

- Thêm InfluxDB vào Data Source:



XV. Tạo dashboard tối thiểu:

- Dashboard tối thiểu gồm có: temperature line chart, humidity line chart, cpu usage chart, memory usage chart:

- + Query temperature:

from(bucket: "iot_data")

|> range(start: -15m)

|> filter(fn: (r) => r["_measurement"] == "sensor_data")

|> filter(fn: (r) => r["_field"] == "humidity")

- + Query humidity:

```
from(bucket: "iot_data")
  |> range(start: -15m)
  |> filter(fn: (r) => r["_measurement"] == "sensor_data")
  |> filter(fn: (r) => r["_field"] == "humidity")
```

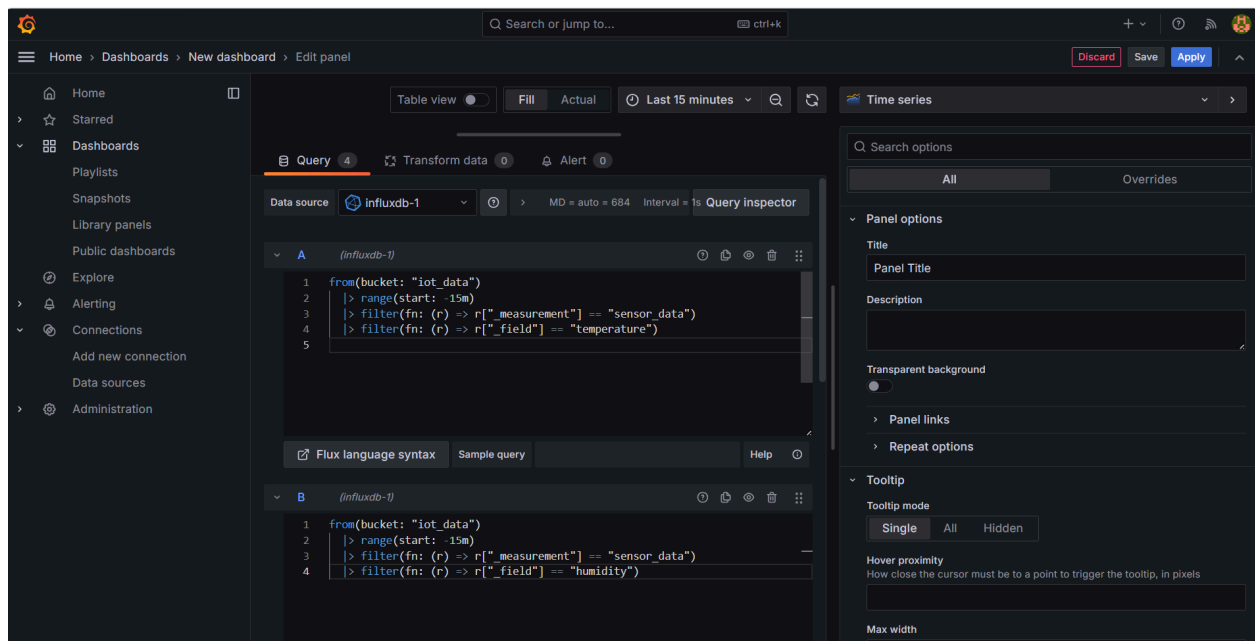
+ Query cpu usage:

```
from(bucket: "iot_data")
  |> range(start: -15m)
  |> filter(fn: (r) => r["_measurement"] == "system_info")
  |> filter(fn: (r) => r["_field"] == "cpu_percent")
```

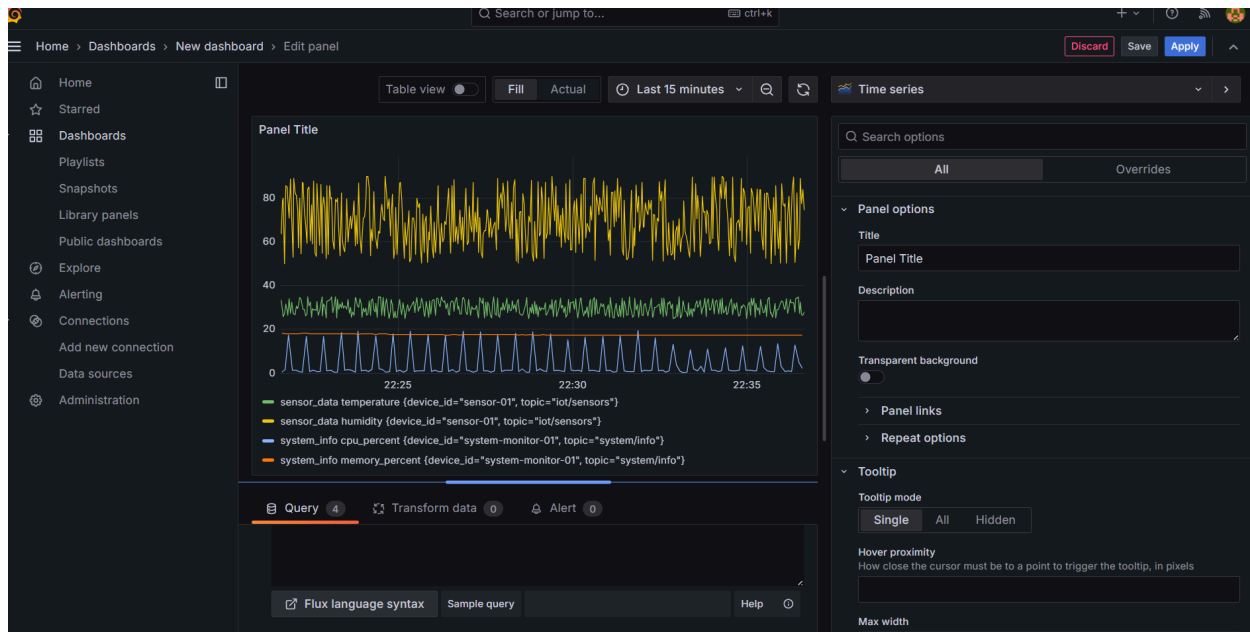
+ Query memory usage:

```
from(bucket: "iot_data")
  |> range(start: -15m)
  |> filter(fn: (r) => r["_measurement"] == "system_info")
  |> filter(fn: (r) => r["_field"] == "memory_percent")
```

- Query: Mỗi trường thông tin là 1 ô query

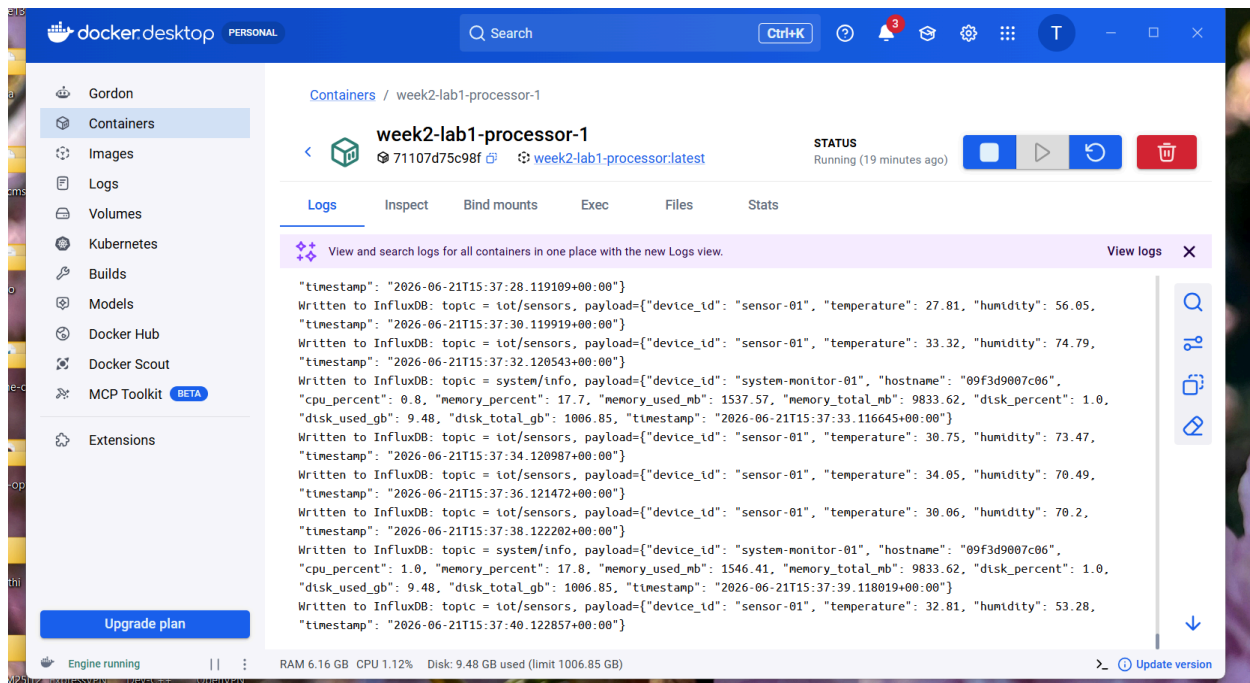


- Kết quả:



XVI. Debug và quan sát hệ thống:

- Các thông tin đã được ghi thành công vào influxDB:



XVII. Câu hỏi báo cáo

1. Docker Compose khác gì so với việc chạy nhiều lệnh `docker run` riêng lẻ?

docker run phải gõ từng lệnh cho từng container, quản lý thủ công thứ tự khởi động, network và biến môi trường. Docker Compose khai báo tất cả trong một file `docker-compose.yml` và khởi chạy toàn bộ hệ thống bằng một lệnh `docker compose up`, tự động tạo network nội bộ giữa các container.

2. Trong docker-compose.yml, vì sao service sensor dùng MQTT_BROKER: mosquitto thay vì localhost?

Bên trong Docker, mỗi container là một máy riêng biệt. localhost sẽ trở về chính container đó, không phải container mosquitto. Docker Compose tự tạo DNS nội bộ, cho phép các container tìm thấy nhau qua tên service — vì vậy dùng mosquitto thay vì localhost.

3. Vai trò của processor trong IoT stack là gì?

Processor đóng vai trò trung gian xử lý dữ liệu: subscribe toàn bộ message từ MQTT broker, parse JSON, chuyển đổi thành định dạng InfluxDB Point rồi ghi vào database. Tách biệt việc thu thập dữ liệu (sensor) khỏi việc lưu trữ (InfluxDB).

4. Vì sao dữ liệu sensor nên được lưu vào time-series database?

Dữ liệu sensor gắn liền với thời gian và được ghi liên tục với tần suất cao. Time-series database như InfluxDB được tối ưu cho việc ghi/đọc theo mốc thời gian, nén dữ liệu hiệu quả, và truy vấn theo khoảng thời gian — khác với database quan hệ thông thường vốn không được thiết kế cho loại dữ liệu này.

5. InfluxDB bucket là gì?

Bucket là đơn vị lưu trữ trong InfluxDB 2.x, tương đương với "database" trong các hệ quản trị CSDL thông thường. Mỗi bucket chứa các measurement (bảng dữ liệu), có thể cấu hình thời gian giữ dữ liệu (retention policy), và được phân quyền truy cập độc lập.

6. Trong Grafana, vì sao URL của InfluxDB phải là <http://influxdb:8086>?

Grafana chạy trong container riêng, không thể dùng localhost để trở đến InfluxDB vì localhost là chính container Grafana. Docker Compose tạo DNS nội bộ nên các container tìm thấy nhau qua tên service — influxdb là tên service khai báo trong docker-compose.yml.

7. Docker volume influxdb-data có vai trò gì?

Lưu trữ dữ liệu của InfluxDB bên ngoài vòng đời container. Khi container bị xóa hoặc restart, dữ liệu vẫn còn nguyên trong volume. Không có volume, mọi dữ liệu sẽ mất khi container dừng.

8. Nếu xóa container InfluxDB nhưng giữ volume, dữ liệu có mất không? Giải thích.

Không mất. Volume tồn tại độc lập với container — xóa container chỉ xóa lớp runtime, không xóa volume. Khi tạo lại container InfluxDB và mount cùng volume đó, toàn bộ dữ liệu cũ sẽ được phục hồi.

9. Nếu muốn thêm một sensor mới, em sẽ sửa hệ thống như thế nào?

Tạo thư mục mới cho sensor (ví dụ `iot-electric-power/`), viết `app.py` publish data lên MQTT topic riêng, viết `Dockerfile`, rồi thêm service mới vào `docker-compose.yml` với `MQTT_BROKER=mosquitto`. Chạy lại `docker compose up -d --build` để khởi động service mới — Docker chỉ build thêm service mới, các service đang chạy không bị ảnh hưởng. Processor tự nhận vì đang subscribe topic # (tất cả), không cần sửa thêm gì.

10. Trong hệ thống thật, vì sao không nên dùng token và password đơn giản như trong bài lab?

Token `iot-token-123` và password `admin123` quá dễ đoán, dễ bị tấn công brute-force hoặc lộ qua log. Trong thực tế cần dùng token ngẫu nhiên độ dài cao, lưu trong secret manager (không hardcode trong file), phân quyền tối thiểu (mỗi service chỉ có quyền vừa đủ), và thay định kỳ.

XVIII. Dọn dẹp môi trường

- Chạy `docker compose down` để dọn service:

```

PS C:\Users\ACER\Desktop\LTAH-IoT\week2-lab1> docker compose down
[+] down 7/7
✓ Container week2-lab1-processor-1      Removed
✓ Container week2-lab1-grafana-1       Removed
✓ Container week2-lab1-system-monitor-1 Removed
✓ Container week2-lab1-iot-sensor-simulator-1 Removed
✓ Container week2-lab1-influxdb-1      Removed
✓ Container week2-lab1-mosquitto-1     Removed
✓ Network week2-lab1_default           Removed
PS C:\Users\ACER\Desktop\LTAH-IoT\week2-lab1>

```

- Chạy docker compose down -v để dọn volumes:

```

PS C:\Users\ACER\Desktop\LTAH-IoT\week2-lab1> docker compose down -v
[+] down 2/2
✓ Volume week2-lab1_grafana_data      Removed
✓ Volume week2-lab1_influxdb_data    Removed
PS C:\Users\ACER\Desktop\LTAH-IoT\week2-lab1>

```

XIX. Kết luận:

Trong bài thực hành này, sinh viên đã triển khai một IoT stack đa dịch vụ bằng Docker Compose. Hệ thống bao gồm MQTT broker, các publisher giả lập dữ liệu, data processor, InfluxDB và Grafana. Đây là bước chuyển quan trọng từ việc chạy container đơn lẻ sang triển khai một hệ thống IoT gồm nhiều dịch vụ có khả năng giao tiếp, lưu trữ và trực quan hóa dữ liệu.